

**Woolworths Online Order and Delivery System –
Enterprise Systems Analysis, Design, and Modelling
Project-II**

Shawn Lee (23204035)

Table Of Content

Summary of Systems Analysis	3
Scope Refinement	3
Refined Static and Dynamic Modelling	4
Refined Class Diagram	4
Sequence Diagram	4
Activity Diagrams / State Machine Diagrams	4
Design Pattern Application and Implementation	4
Application	4
Implementation	5
Comparative Design Analysis	5
Critical Reflection	6
Design Pattern Tradeoffs	6
Limitations of the final design and implementation	6
Lessons learned	6
References	7
Appendices	7

Summary of Systems Analysis

From the analysis of part 1, a proposal has been conducted for Woolworths New Zealand's Online and Delivery System. Based on some assumptions made. The previous report analyzed the identified key operational problems across 3 departments. These departments include store operations, supply chain & Logistics, and customer service.

The main issue found during the system analysis is that there is currently a heavy reliance on manual processes, which reduces real-time data visibility and efficiency in staff communication. This information is collected through multiple sources, which include the original problem statement, interviews with stakeholders, and, lastly, analyzing legacy systems. This directly helped produce the business goals, user requirements, functional requirements, and non-functional requirements.

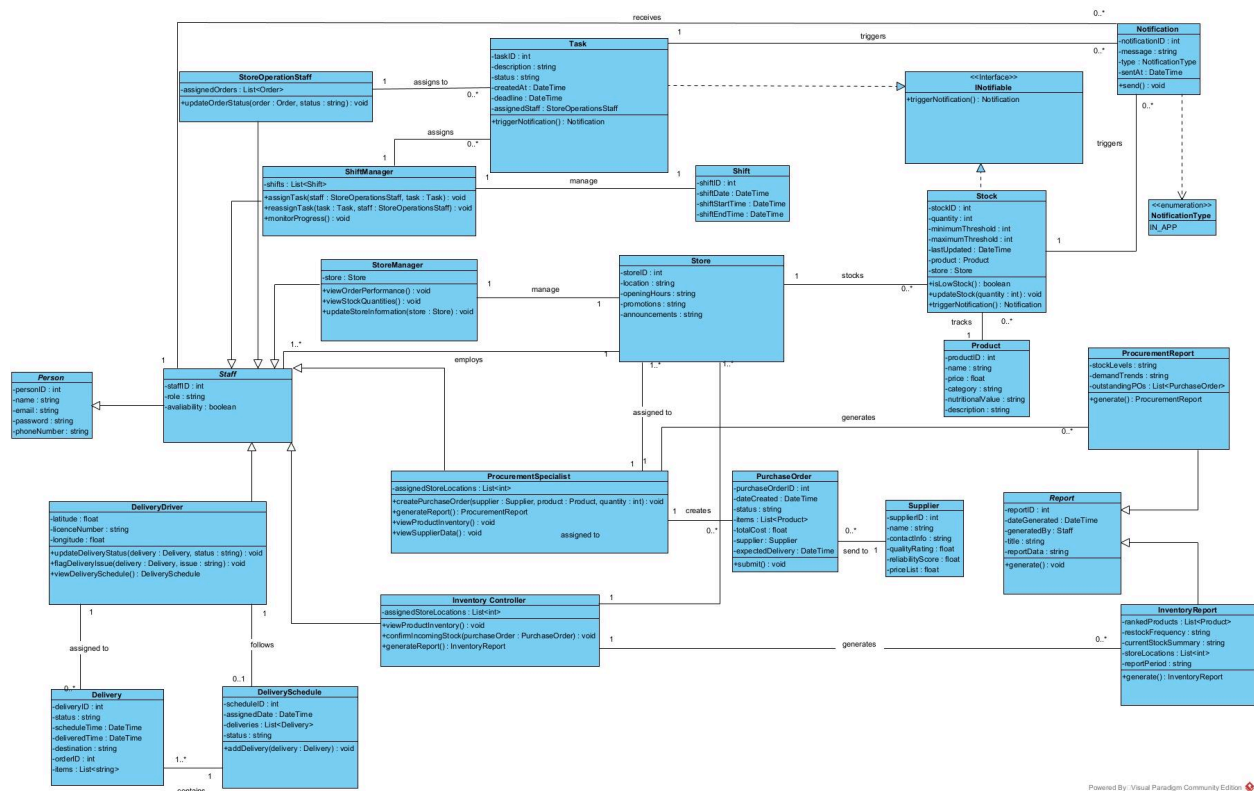
In part 1, both the initial class diagram and use case diagram are created. The use case diagram includes both the customer-facing use case and another use case diagram for the internal management system. The initial class diagram covers classes related to customer, staff order, inventory, delivery, and reporting. Upon creating this, several weaknesses are found in the class diagram, including the absence of abstract classes and interfaces, and redundancies.

Scope Refinement

Continuing this report, the scope has been reduced to just focusing on the internal management system, and the customer-facing Online platform is out of the scope. Customer and customer service representative functionality has been excluded due to time constraints. Additionally, delivery management has been moved from the supply chain & logistics department towards the customer service department. The scope covered within the internal management system would be the Store operation staff, Shift manager, store manager, procurement specialist, inventory controller, and delivery driver.

Refined Static and Dynamic Modelling

Refined Class Diagram



This updated iteration of the class diagram shows:

- **Abstract classes of Person, Staff, and Report (Italic)**

Since these classes are never initialized directly, Person and Staff are base classes for the subclasses of staff. Report is the base class for the Procurement report and the Inventory report class.

- Enum NotificationType

Changed from a string attribute called "type" to adding an enum "NotificationType", even though there is just 1 constant value in NotificationType, it would be helpful for future different types of notifications, for example, email or SMS.

- **Changed currentLocation**

From the risk identified from phase 1, the `currentLocation` variable in the `DeliveryDriver` class is split into `Longitude` and `latitude` to enable exact distance calculations for delivery scheduling.

- **Added Interface**

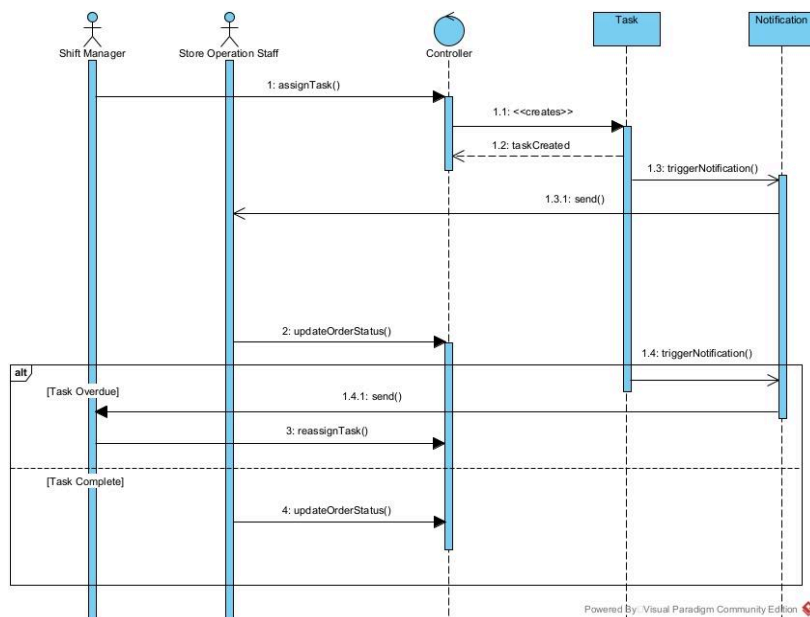
An interface has been added to enforce methods on multiple classes. The interface called “`INotifiable`” is used for enforcing the `triggerNotification` method on both the `Task` and `Stock` classes.

- **Scope Reduced**

As stated in the Scoper refinement, there will be no customer-facing functionalities, thus customer, cart, shoppingList, ShoppingListItem, Customer service representative, complaints, and refund.

Sequence Diagram

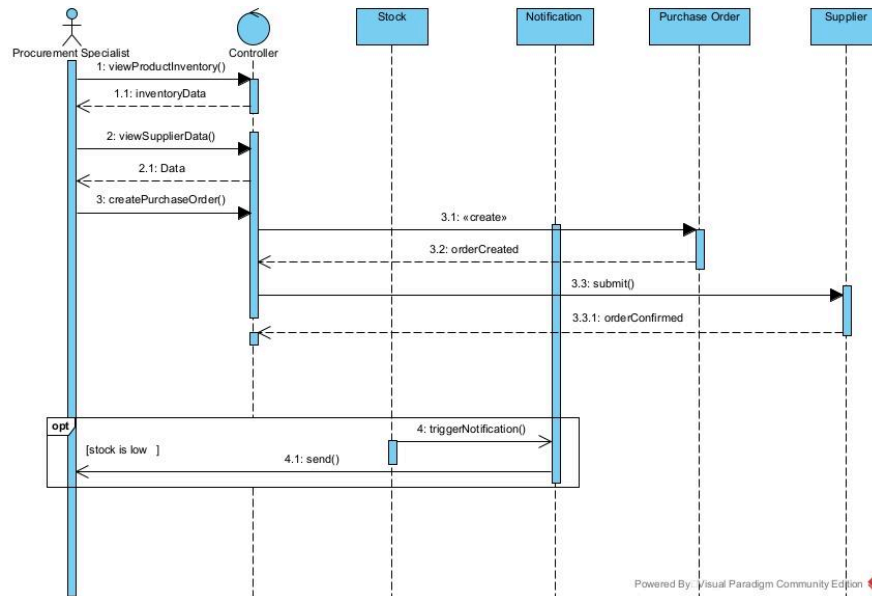
Assign Task Sequence Diagram



This sequence diagram shows how the shift manager allocates tasks to the store operations staff for picking and packing. The process starts when the Shift Manager lifeline calls the `assignTask()` method, which sends a request to the controller lifeline, which initialises a task object. Once the task object is created, the task object calls the `triggerNotification()` method asynchronously towards the notification lifeline. This triggers the `send()` method, which is how store operation staff are notified of a new task to complete.

The store operation staff calls the `updateOrderStatus()` method when there are changes to the pick and pack task. An `alt`-combined fragment is used to show the alternate flow of whether the task is overdue or completed. The task overdue flow shows the task lifeline calling the `triggerNotification()` method, which notifies the shift manager. The shift manager then calls the `reassignTask()` method. The task completion flow just shows calling the `updateOrderStatus()` to change the status to completed.

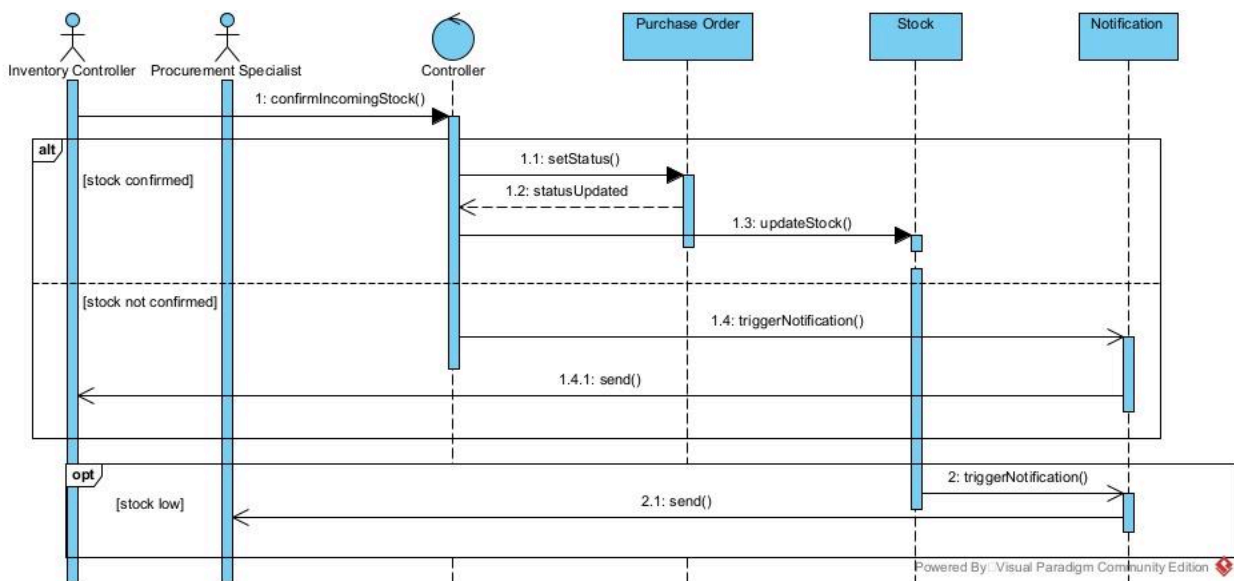
Low Stock Notification and Purchase Order Sequence Diagram



This sequence diagram shows the sequence of events occurring during the procurement specialist's purchasing of orders from suppliers. It begins with the procurement specialist calling the `viewProductInventory()` method towards the controller lifeline, which returns inventory data. Then the procurement specialist calls the `viewSupplierData()` method on the controller to return data to gain more information about which supplier to choose. Once a supplier is chosen, a new purchase order object is initialised, and the `submit()` method is called towards the supplier, which returns the order statement.

The opt-combined fragment shown is the optional flow when the `isLowStock()` method returns true, which triggers the `triggerNotification()` method asynchronously. The `send()` method is called to notify the procurement specialist.

Confirm Incoming Stock Sequence Diagram

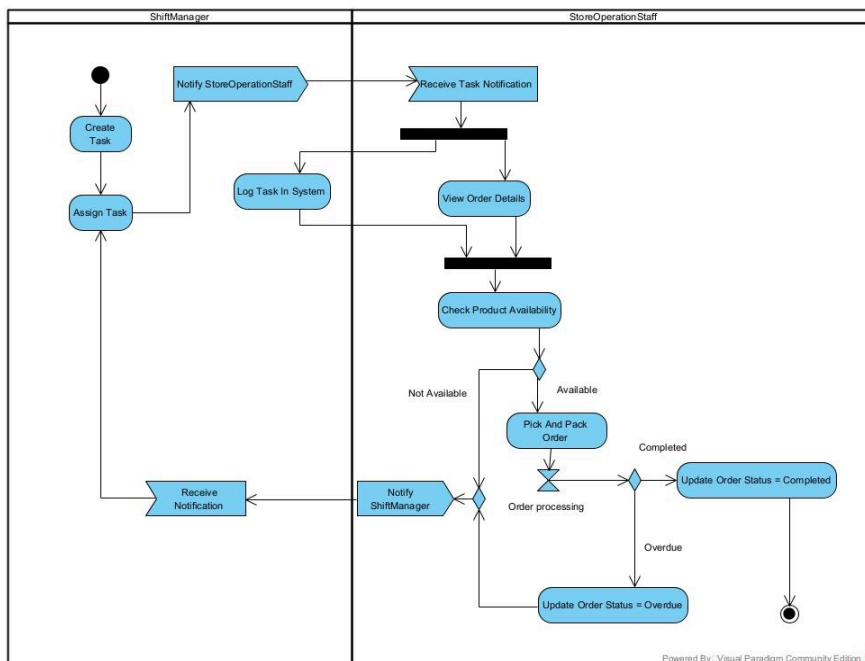


This sequence diagram depicts the process of an inventory controller checking incoming stock from a supplier shipment and immediately notifying the procurement specialist of the result. The sequence begins with the inventory controller calling the `confirmIncomingStock()` method on the controller. An alternate combination fragment has two flows. The stock-confirmed flow is when the purchase order is completed, which then calls the `setStatus()` method, returns the status, and then calls the `updateStock()` method. The stock not confirmed flow calls the `triggerNotification()` method on the Notification object, which calls the `send()` method to notify the inventory controller.

As stated in the Low Stock Notification and Purchase Order Sequence Diagram explanation, the optional combined fragment is the same in the Confirm Incoming Stock Sequence Diagram.

Activity Diagrams / State Machine Diagrams

Assign and Monitor Task Activity Diagram



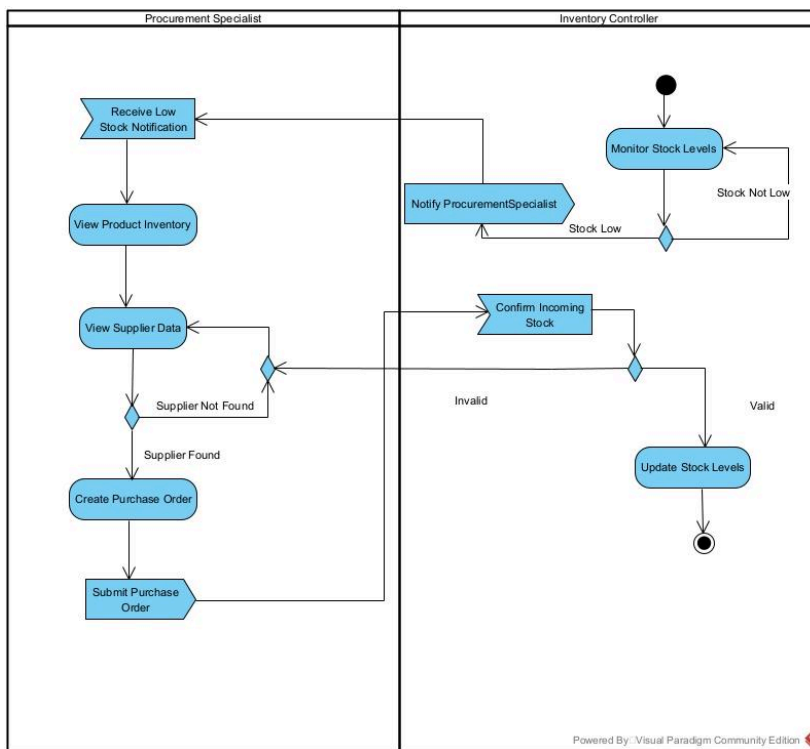
This activity diagram illustrates the flow of actions between the shift manager and the store operations staff during task assignment and monitoring.

Starts off the flow of actions with the creation of a task, assigns it, and then notifies the store operation staff. A forked node is used to represent the log task in the system action and the view order detail action in parallel. The joint node is used to synchronise

both actions, as both actions need to be completed before flowing to the next action.

There is a decision node after the 'Check product availability' action, since the product could be available or not, depending on the guard conditions. If not available, flows to the merge node to notify the shift manager of the issue. If available, process the pick and pack order action in which a time event occurs. A decision node condition is shown on the diagram after the time event, which checks whether the task is overdue or completed on time, as are the guard conditions. If the task is overdue, the store operation staff updates the order status to overdue. This merges into the notify ShiftManager action, which loops back to the assign task action. If the task is completed on time, the store operation staff updates the order status to completed, and the process ends (activity final).

Purchase Order Process



This activity diagram shows the process of managing stocks and creating purchase orders during situations where the stock is low.

The inventory controller monitors stock levels. A decision node checks for Stock not low or Stock low. If the stock is low, the inventory controller tells the procurement specialist. This is shown by sending and receiving a signal.

The procurement specialist views product inventory and supplier data. A decision node checks for a suitable supplier. If none is found, the flow returns to View Supplier Data. If a supplier is chosen, a purchase order is created and submitted for validation.

The flow crosses back to the inventory controller, and the Confirm Incoming Stock action receives the submission. There is a decision node that checks if the purchase order is valid or not. If invalid, loops back towards the action of viewing supplier data. If valid, it allows the stock level to update and ends the activity diagram.

Design Pattern Application and Implementation

Application

- ☐ Application of at least two GoF design patterns
- ☐ UML diagrams showing pattern structure

Implementation

- ☐ Corresponding C# implementation demonstrating each pattern

Comparative Design Analysis

Students must provide a critical comparison of the system design before and after the application of design patterns, addressing the following aspects:

- ☐ Changes to class structure and responsibility allocation
- ☐ Improvements in cohesion and reductions in coupling
- ☐ Impact on extensibility and maintainability
- ☐ Consequences for C# implementation complexity
- ☐ Examples of how new requirements can be accommodated more easily

The analysis must be supported by UML diagrams and concrete references to the C# codebase.

Critical Reflection

Design Pattern Tradeoffs

- ☐ Design trade-offs introduced by design patterns

Limitations of the final design and implementation

- ☐ Limitations of the final design and implementation

Lessons learned

- ☐ Lessons learned from aligning UML models with C# code.

References

Appendices

- GitHub
(<https://github.com/ShawnLeeyz/Woolworths-Systems-Analysis-Design-and-Modelling-Project-I.git>)